FleetOps V2

Cloud Native Fleet Management Platform

AWS EKS | Terraform | GitHub Actions | ArgoCD

🏅 GitHub Actions Certified · GH200

UST Fresher Capstone • June 2026

fleetops.website

FLEETOPS V2

Project Overview

Production-grade microservices platform — vehicle tracking, maintenance scheduling, service requests, and AI-powered advisory.

🌐 fleetops.website

4	27	3	5+
Spring Boot 3.x Microservices	Terraform Modules	CI/CD Pipelines	Bonus Features

- Amazon EKS 1.31 (us-east-1) for container orchestration
- Terraform IaC with 27 modular configurations
- GitOps delivery via ArgoCD + GitHub Actions
- IRSA for zero static credential access
- Amazon Nova Lite (amazon.nova-lite-v1:0) AI integration via Bedrock

THE PROBLEM

Why FleetOps?

The Problem

📍 No fleet visibility

Managers call drivers individually to know vehicle status

🔁 No service request lifecycle

Repairs tracked verbally — no records, no accountability

🗓️ Deadlines missed

Insurance renewals & service dates lost in shared Excel files

🔧 Tasks fall through

Work assigned verbally — invisible when a mechanic is absent

📁 Documents scattered

Inspection photos emailed around, never linked to a vehicle

What FleetOps Delivers

- ✅ **Vehicle registry**
  Status · mileage · driver — live dashboard KPIs
- ✅ **Step Functions workflow**
  Raised → Assigned → In Progress → Completed
- ✅ **Lambda daily scanner**
  Scans every vehicle → SNS alert → email to managers
- ✅ **Per-user task queue**
  Technicians see their work; managers assign across the team
- ✅ **Shared EFS + S3**
  Photos & PDFs tagged to vehicle, accessible fleet-wide
- ✅ **AI fleet analysis**
  Bedrock Nova Lite — health score + recommended actions

01

Infrastructure as Code

27 Terraform Modules | Modular | State-Managed

SECTION 1 — IaC

Terraform Architecture

27 Modules Total

Networking	VPC · 3 Subnet Tiers · NAT Gateways · Multi-AZ
Compute	eks/cluster · eks/nodegroup · eks/addons · eks/oidc
Data	RDS · Redis · S3
Messaging	SNS · SQS · EventBridge · Step Functions · Lambda
Security	IAM · KMS · Secrets Manager · SSM · WAF
Delivery	ECR · ACM · Route53 · ALB · CloudFront
Observability	CloudWatch · CloudTrail · Config
AI / Bedrock	Amazon Bedrock — Nova Lite (amazon.nova-lite-v1:0) · Cross-account IAM

SECTION 1 — IaC

Terraform Code Quality Standards

✓ **Modular Structure**

modules/networking, modules/eks, modules/database, modules/storage

✓ **Remote State**

S3: fleetops-terraform-state-johan + DynamoDB: fleetops-terraform-locks

✓ **No Hardcoded Values**

All config in terraform.tfvars per environment (dev / staging / prod)

✓ **All Resources Tagged**

Environment and Owner tags on every resource

✓ **.gitignore Complete**

Excludes *.tfstate, .env, *.tfvars, credentials

✓ **Outputs Exported**

Cluster endpoint, OIDC URL, ECR URIs — all exported

✓ **Code Clean** terraform validate and terraform fmt — both pass

02

Kubernetes Deployment

5 Services · HPA · Health Probes · Network Policies

SECTION 2 — K8S

Kubernetes Deployment Architecture

Namespace: fleetops-prod (not default)

auth-service	vehicle-service	maintenance-service	request-service	frontend (nginx)
2+ Replicas	2+ Replicas	2+ Replicas	2+ Replicas	2+ Replicas
JWT auth, stateless	Bedrock AI + S3	SNS publishing	Step Functions	React 18 SPA

Every service includes:

Deployment | Service | HPA | ConfigMap | ExternalSecret

Liveness: httpGet /health **Readiness:** httpGet /health (all 5 services)

Java services — requests: cpu 100m / mem 256Mi limits: cpu 500m / mem 512Mi

Frontend (nginx) — requests: cpu 100m / mem 128Mi limits: cpu 250m / mem 256Mi

SECTION 2 — K8S

GitOps with ArgoCD

App-of-Apps Pattern

📦 **root-app-prod.yaml**

- ├— auth-prod.yaml
- ├— vehicle-prod.yaml
- ├— maintenance-prod.yaml
- ├— request-prod.yaml
- ├— frontend-prod.yaml
- ├— secrets-prod.yaml ← ExternalSecrets
- ├— ingress-prod.yaml ← ALB
- ├— networkpolicy-prod.yaml ← micro-segmentation

Sync Configuration

Sync Policy	**Automated**
Prune	**Enabled**
Self Heal	**Enabled**
Sync Latency	**~90 sec**

ArgoCD Dashboard: argocd.fleetops.website

03

CI/CD Pipelines

3 GitHub Actions Workflows · Security Gates

SECTION 3 — CI/CD

Three GitHub Actions Workflows

Build Pipeline

java-ci-ecr.yml

push to main / develop

prepare → quality → build → trivy-scan → push → release → notify

Deploy Pipeline

cd-ecr.yml

successful build

update-dev → update-prod → smoke-test → notify

Infrastructure Pipeline

terraform-apply.yml

push main: environments/**, modules/** + workflow_dispatch

validate + checkov → plan → manual approval → apply

Security Scanning in CI/CD

Gate 1

Dependency Scanning

Tool: Snyk

Third-party libraries
& open-source packages

Gate 2

SAST / Code Quality

Tool: SonarQube

Static analysis, code
quality & security rules

Gate 3

Container Scanning

Tool: Trivy

Docker image CVEs
& misconfigurations

OIDC-based AWS credentials — Zero static keys in GitHub Secrets

04

Cloud Integration

IRSA · Bedrock AI · S3 · SNS · Step Functions

SECTION 4 — CLOUD

IRSA — Zero Static Credentials

1

Terraform — OIDC Provider

```
resource "aws_iam_openid_connect_provider" "eks" { ... }
```

modules/eks/oidc creates the OIDC provider linked to EKS cluster

2

IAM Role with OIDC Trust Policy

```
"Federated": "arn:aws:iam::538661800892:oidc-provider/{url}"
```

Trust policy scoped to fleetops-prod:fleetops-app service account

3

ServiceAccount Annotation

```
eks.amazonaws.com/role-arn: arn:aws:iam::...:role/fleetops-prod-app-irsa-role
```

Kubernetes ServiceAccount annotated — pods receive automatic token exchange

What Pods Can Access (IRSA Permissions)

Permission	Resource	Service Using It
S3 Get/Put/Delete	vehicle-docs bucket	vehicle-service
Bedrock InvokeModel	foundation-model/*	vehicle-service
SNS Publish	insurance-alerts, service-alerts	maintenance-service
Step Functions Start/Describe	Request workflow	request-service
Secrets Manager	fleetops/*	All services (via ESO)
CloudWatch Logs	/fleetops/* log groups	All services
KMS Decrypt	All KMS keys	All services

Zero static credentials for AWS services — IRSA handles all native AWS access

Amazon Nova Lite AI Integration

🤖 **Fleet Maintenance Advisor — FleetAiService.java · vehicle-service only**

Model:	amazon.nova-lite-v1:0 (Amazon Nova Lite via Bedrock Converse API)
Use Case:	Analyses all vehicles → returns health score + maintenance recommendations
Auth:	Cross-account IAM credentials in Secrets Manager → K8s Secret fleetops-bedrock-secret
Caching:	Redis 15-min cache @Cacheable("fleetAnalysis") — avoids redundant Bedrock calls
Audit:	Every call saved to ai_analysis_audit table (model, requester, health score, exec time)
Endpoint:	GET /api/vehicles/ai/fleet-analysis

⚠️ Note: Terraform IAM policy lists claude-3-haiku / claude-v2 — stale; actual runtime model is nova-lite-v1:0

05

Security & Compliance

All Requirements Met — Zero Hardcoded Credentials

SECTION 5 — SECURITY

Security Checklist — All Met

✓	**No hardcoded credentials**	External Secrets Operator + IRSA	✓	**Non-root containers**	spring:spring user (Java), nginx user (frontend)
✓	**Multistage Docker builds**	All 5 services — Alpine base, JRE-only final image	✓	**Container image scanning**	Trivy in CI — blocks on HIGH/CRITICAL
✓	**Secrets in K8s Secrets**	ExternalSecrets pulls from AWS Secrets Manager	✓	**RBAC configured**	ServiceAccount minimal permissions via IRSA
✓	**Encryption at rest**	KMS for RDS, S3, SNS, SQS, Secrets Manager	✓	**Audit trail**	CloudTrail to encrypted S3
✓	**WAF**	AWS WAFv2 Common Rule Set on CloudFront	✓	**.gitignore complete**	Excludes *.tfstate, .env, *.tfvars, credentials

SECTION 6 — BONUS

Bonus Features Achieved

AI Integration

Amazon Nova Lite via Bedrock Converse API

Event-Driven Architecture

EventBridge → Lambda, SNS, SQS + DLQ, Step Functions (6 states)

GitOps Deployment

ArgoCD App-of-Apps, automated sync, selfHeal=true

Network Policy

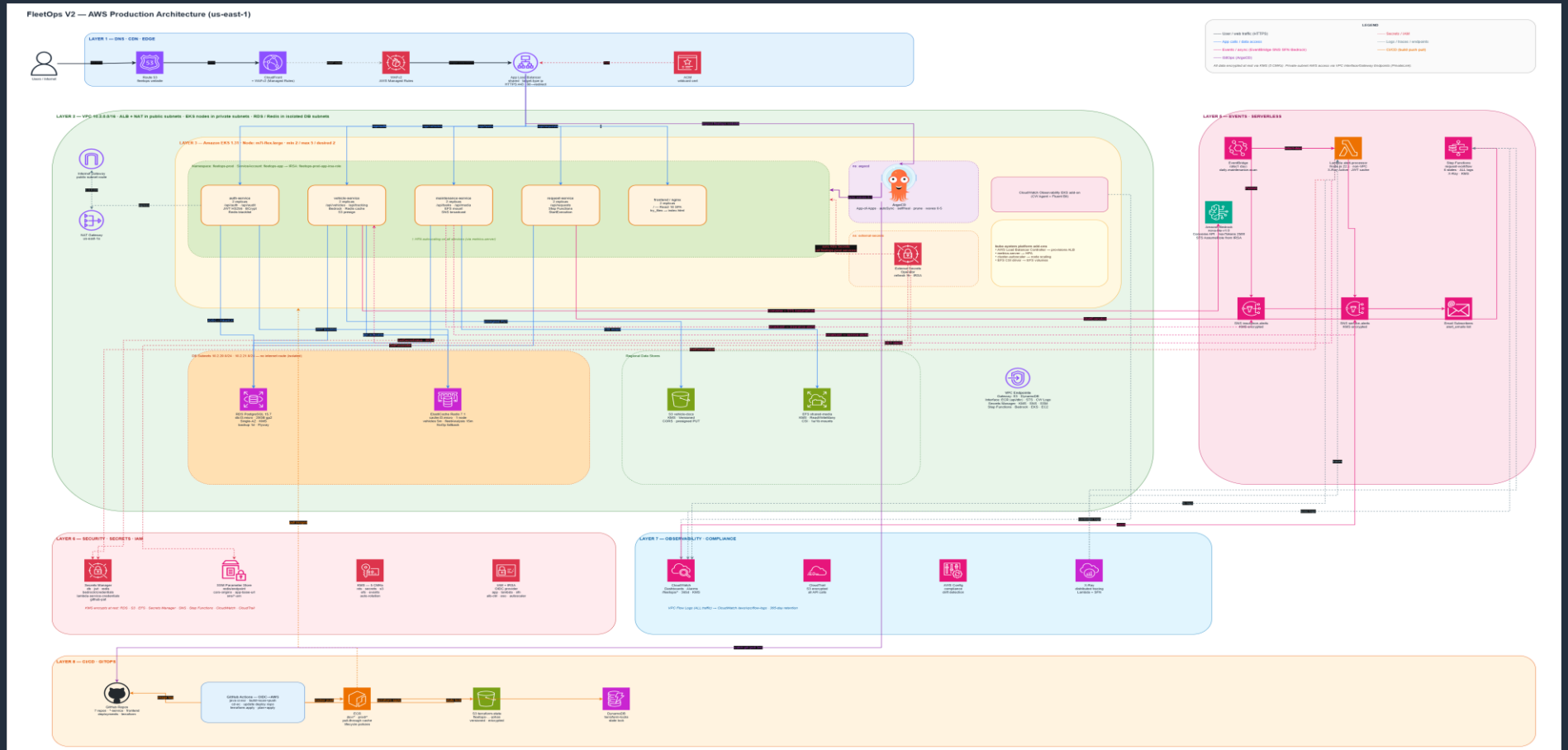
allow-apps-to-rds.yaml, allow-service-discovery.yaml

Advanced Monitoring

CloudWatch dashboards + metric alarms (RDS, ALB, Lambda)

AWS PRODUCTION ARCHITECTURE — us-east-1

DNS / CDN / Edge · VPC / Subnets · EKS · Events / Serverless · Security · Observability · CI/CD & GitOps



APPLICATION ARCHITECTURE

7 Layers · Microservices · Auth & RBAC · Service-to-Service · State Machine · Alert Flows

1 LAYER 1 – CLIENT / ENTRY POINTS

Browser Client

React SPA (Vite)
Served by
nginx (frontend/nginx pod)

JWT stored in HttpOnly cookie (SameSite=Strict) — set by auth-service on login

All API calls →
https://fleetops.website/api/*
via CloudFront → ALB

Lambda alert-processor (Node.js 22.x)
- Not in VPC
- Calls via https://origin.fleetops.website/api/*
- Uses Authorization: Bearer <token> header (no cookie)

CloudFront → WAFv2 → Application Load Balancer

2 LAYER 2 – CROSS-CUTTING: AUTH & RBAC

JWT Auth Flow (all 4 Java services)

1. POST /api/auth/login → auth-service generates JWT (HMAC, shared jwt.secret)
2. Browser: receives HttpOnly cookie jwt=<token> + body {username, role, token}
3. Lambda/service-to-service: uses Authorization: Bearer <token> header
4. Every service validates JWT locally (no auth-service call per request)

Public endpoints (no JWT required)
- /api/auth/login
- /api/auth/register
- /api/auth/logout
- /actuator/health
- /health

Roles (3 levels)

ADMIN — Full access (all endpoints)

MANAGER — Read all + status updates + approvals + alerts + AI analysis + alarm broadcast + live tracking

DRIVER — Own vehicle only + own requests + own tasks + mileage update + media upload + tracking ping

3 LAYER 3 – MICROSERVICES

A auth-service	B vehicle-service	C maintenance-service	D request-service
Spring Boot 3.5 Java 21 | Port 8080	Spring Boot 3.5 Java 21 | Port 8080	Spring Boot 3.5 Java 21 | Port 8080	Spring Boot 3.5 Java 21 | Port 8080
Auth & User Management • Register / Login / Logout • Me (profile) • Audit Events	**Vehicle, Documents, Tracking, Alerts & AI Analysis** • Vehicle CRUD & status • Document management (S3) • GPS tracking (ping, live, history) • Alerts (insurance/service) • Fleet dashboard & AI analysis	**Task Queue, Alarms, Shared Media** • Maintenance task queue • Alarm broadcast (insurance + service alerts) • Shared media management	**Service Requests & Workflow Orchestration** • Service requests CRUD (no create for MANAGER) • State transitions & rules • Technician assign / dispatch • Vehicle status sync • Step Functions integration
DB: RDS PostgreSQL users table	DB: RDS PostgreSQL vehicles, vehicle_telemetry	DB: RDS PostgreSQL maintenance_queue, pending_tasks	DB: RDS PostgreSQL service_requests (stepFunctionsExecutionArn)

Cache: Redis (ElastiCache) — vehicles + fleetAnalysis cache, 5-min TTL, @Cacheable/@CacheEvict
Storage: Amazon S3

Storage: Amazon EFS /var/www/fleetops/shared-media (KMS encrypted)

4 LAYER 4 – SERVICE-TO-SERVICE CALLS

request-service	→ vehicle-service	GET	/api/vehicles/{id}	(validate vehicle on all ops)
	→ vehicle-service	PATCH	/api/vehicles/{id}/status	(sync status, 3 retries)
	→ maintenance-service	POST	/api/tasks/add	(on technician assign)
	→ AWS Step Functions	StartExecution		(on request create)
maintenance-service	→ vehicle-service	GET	/api/vehicles/alerts/insurance	(on broadcast)
	→ vehicle-service	GET	/api/vehicles/alerts/service	(on broadcast)
	→ Amazon SNS	Publish to both topics		(on broadcast)
Lambda alert-processor	→ auth-service	POST	/api/auth/login	(get JWT, cached, refresh 5min before expiry)
	→ vehicle-service	GET	/api/vehicles/alerts/insurance	
	→ vehicle-service	GET	/api/vehicles/alerts/service	
	→ Amazon SNS	Publish to insurance-alerts + service-alerts		
vehicle-service	→ Amazon Bedrock	InvokeModel (Nova Lite)		(fleet AI analysis)
	→ Amazon S3	GetPresignedUrl / ListObjects		

All K8s internal calls use http://fleetops-<service>:8080 (internal DNS)

Lambda & external calls use https://origin.fleetops.website/api/*

5 LAYER 5 – DATA STORES PER SERVICE

Service	Store	Purpose
auth-service	Amazon RDS PostgreSQL	users (id, username, email, password_hash, role)
vehicle-service	Amazon RDS PostgreSQL	vehicles, vehicle_telemetry
	Amazon ElastiCache Redis	vehicles + fleetAnalysis cache 5-min TTL, @Cacheable/ @CacheEvict
	Amazon S3	Vehicle documents, presigned PUT URLs
maintenance-service	Amazon RDS PostgreSQL	maintenance_queue, pending_tasks
	Amazon EFS	Shared media files /var/www/fleetops/ shared-media
request-service	Amazon RDS PostgreSQL	service_requests (includes stepFunctionsExecutionArn)

6 LAYER 6 – AWS INTEGRATIONS PER SERVICE

Service	AWS Service	How / Purpose
vehicle-service	Amazon Bedrock (Nova Lite)	Via IRSA, invoked by FleetAiService
	Amazon S3	Via IRSA, presigned URL + list
maintenance-service	Amazon SNS	Via IRSA, SDK publish to both topics (insurance-alerts, service-alerts)
request-service	AWS Step Functions	Via IRSA, StartExecution on request create
Lambda	AWS Secrets Manager	GetSecretValue for credentials
	auth-service → SNS	Full flow: login → get alerts → publish to both topics
All services	Amazon CloudWatch Logs	Audit trail → /fleetops/audit-trail via AuditController
	AWS X-Ray	Lambda + Step Functions tracing enabled

7 REQUEST-SERVICE – STATE MACHINE & BUSINESS RULES

State Machine (enforced in code)

OPEN → PENDING_APPROVAL → APPROVED → ASSIGNED → IN_PROGRESS → COMPLETED
PENDING_APPROVAL → REJECTED

Terminal states: COMPLETED, REJECTED
No further transitions allowed

State Transitions → Vehicle Status Sync
- Create:
  - vehicle → IN_SERVICE
  - (or BREAKDOWN for BREAKDOWN type)
- IN_PROGRESS: vehicle → IN_SERVICE
- COMPLETED or REJECTED: vehicle → ACTIVE
- On ASSIGN: calls POST /api/tasks/addTusername={technician} on maintenance-service (creates task in technician's queue)

Step Functions (Audit / Tracing Only)
- Starts on every new request (on create)
- Stores executionArn in service_requests.step_functions_execution_arn
- All 6 states are Pass type (instant)
- Input: {requestId, vehicleId, requestType, requestedBy}

Request Type Rules (Server-Side)
- BREAKDOWN → priority HIGH, vehicle status → BREAKDOWN immediately
- INSURANCE_RENEWAL → only allowed within 30 days of expiry
- ROUTINE_SERVICE → only when service due by date or mileage
- All others → vehicle status → IN_SERVICE on create

END-TO-END ALERT & NOTIFICATION FLOWS (SUMMARY)

Lambda Daily Alert Flow (Automated)

EventBridge (Rate: 1 day) → Lambda alert-processor → Secrets Manager (credentials) → auth-service POST /api/auth/login (get JWT) → vehicle-service GET /alerts/insurance, GET /alerts/service → Amazon SNS Topics (KMS Encrypted) → Email Subscribers

Amazon SNS Topics (KMS Encrypted): insurance-alerts (insurance expiry), service-alerts (service expiry + info alarms)

Manager (Manual) → maintenance-service POST /api/tasks/alarms/broadcast → vehicle-service GET /alerts/insurance, GET /alerts/service → Amazon SNS Topics (KMS Encrypted) → Email Subscribers

ROLE-BASED ACCESS MATRIX (SUMMARY)

Capability	ADMIN	MANAGER	DRIVER
User & Role Management	✓ Full	✓	✗
Vehicle Management	✓ Full (CRUD)	✓ View/Update Status	✓ Own Only
Requests	✓ Full	✓ View/Manage (no create)	✓ Own Only
Task Queue	✓ Full	✓ Full	✓ Own Only
Media Management (EFS)	✓ Full	✓ Full	✓ Full
Alerts & AI Analysis	✓ Full	✓ Full	✗
Alarm Broadcast	✓ Full	✓ Full	✗
Live Tracking	✓ Full	✓ Full	✓ Ping + Position Lookup
Dashboard / Reports	✓ Full	✓ Full	✗

✓ = Allowed ✗ = Not Allowed

Thank You

FleetOps V2 — Production-grade fleet management platform
Built end-to-end as a DevOps Capstone

Repository: github.com/FleetOps-V2

Live Site: fleetops.website

ArgoCD: argocd.fleetops.website

Questions welcome.